

Team 201 - Actor-Critic

August 13, 2025

0.0.1 Group ID: 201

0.0.2 Group Members Name with Student ID:

1. JAGADI SHRUTI CHANABASAPPA: 2024AA05181
2. TANAY KASHYAP: 2024AA05991
3. SHIKHAR NIGAM: 2024AA05691
4. KHUSHWANT SINGH RATHORE: 2024AA05490

0.1 Objective:

The goal of this assignment is to model the ICU treatment process using Reinforcement Learning, specifically the Actor-Critic method. The agent should learn an optimal intervention policy from historical ICU data. Each patient's ICU stay is treated as an episode consisting of time-stamped clinical observations and treatments. Your tasks: 1. Model the ICU treatment process as a Reinforcement Learning (RL) environment. 2. Train an Actor-Critic agent to suggest medical interventions based on the patient's current state (vitals and demographics).

0.2 Dataset:

Use the dataset provided in the following link:

<https://drive.google.com/file/d/1UPsOhUvyrsrC59ilXsvHwGZhzM7Yk01w/view?usp=sharing>

Features:

- *Vitals*: mean_bp, spo2, resp_rate
- *Demographics*: age, gender
- *Action*: Medical intervention (e.g., "Vancomycin", "NaCl 0.9%", or NO_ACTION)
- *Identifiers*: timestamp, subject_id, hadm_id, icustay_id

0.3 State Space :

Each state vector consists of: mean_bp (Mean Blood Pressure) , spo2 (Oxygen Saturation), resp_rate (Respiratory Rate), age, One-hot encoded gender

0.4 Action Space :

- The agent selects one discrete action from 99 possible medical interventions (e.g., Vancomycin, Fentanyl, PO Intake, etc).

- You should integer encode or one-hot encode these interventions.

0.5 Reward Function:

At each time step, the agent receives a reward based on how close the patient's vitals are to clinically normal ranges. The reward encourages the agent to take actions that stabilize the patient's vital signs:

$$\text{Reward}_t = -((MBP_t - 90)^2 + (SpO2_t - 98)^2 + (RR_t - 16)^2)$$

Explanation:

- MBP (mean_bp): Target = 90 mmHg
- SpO (spo2): Target = 98%
- RR (resp_rate): Target = 16 breaths/min

Each term penalizes the squared deviation from the healthy target. The smaller the difference, the higher (less negative) the reward.

Example:

Suppose at time t, the vitals are:

- MBP = 88
- SpO = 97
- RR = 20

Then the reward is:

$$\text{Reward}_t = -[(88 - 90)^2 + (97 - 98)^2 + (20 - 16)^2] = -(4 + 1 + 16) = -21$$

A lower (more negative) reward indicates worse vitals, guiding the agent to learn actions that minimize this penalty.

0.5.1 Episode Termination

An episode ends when the ICU stay ends. To define this:

1. **Group the data** by `subject_id`, `hadm_id`, and `icustay_id`
→ Each group represents one ICU stay = one episode.
2. **Sort each group** by `timestamp`
→ Ensures the time progression is correct.
3. **For each time step** in a group (i.e., each row):
→ Check if it is the **last row** in that group.
 - If **yes**, then mark `done = True` (end of episode)
 - If **no**, then mark `done = False` (continue episode)

0.6 Requirements and Deliverables:

Implement the Sepsis Treatment Optimization Problem for the given above scenario for the below mentioned RL method.

0.6.1 Initialize constants

```
[2]: import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.preprocessing import LabelEncoder, StandardScaler
import matplotlib.pyplot as plt
from collections import defaultdict
import warnings
warnings.filterwarnings('ignore')

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# Hyperparameters
LEARNING_RATE_ACTOR = 0.001
LEARNING_RATE_CRITIC = 0.002
DISCOUNT_FACTOR = 0.99
ENTROPY_BETA = 0.01
MAX_EPISODES = 1000

# State/Action dims
STATE_DIM = 5
ACTION_DIM = 99

# Reward targets
TARGET_MBP = 90
TARGET_SPO2 = 98
TARGET_RR = 16

print("TensorFlow version:", tf.__version__)
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

TensorFlow version: 2.19.0

GPU Available: []

0.6.2 Load Dataset (0.5 Mark)

```
[3]: def load_and_preprocess_data():
    """Load dataset and prepare for RL environment."""
    # Load CSV
    df = pd.read_csv("Sepsis_datset.csv")
```

```

# Convert timestamps to datetime
df['timestamp'] = pd.to_datetime(df['timestamp'])

# Sort by ICU stay and timestamp
df = df.sort_values(['subject_id', 'hadm_id', 'icustay_id', 'timestamp'])

# Encode gender (F=0, M=1)
gender_encoder = LabelEncoder()
df['gender'] = gender_encoder.fit_transform(df['gender'])

# Encode actions as integers
action_encoder = LabelEncoder()
df['action'] = action_encoder.fit_transform(df['action'])

print("Dataset shape:", df.shape)
print("Unique ICU stays:", len(df.groupby(['subject_id', 'hadm_id', 'icustay_id'])))
print("Gender classes:", list(gender_encoder.classes_))
print("Number of unique actions:", len(action_encoder.classes_))

return df, action_encoder, gender_encoder

data, action_encoder, gender_encoder = load_and_preprocess_data()

```

Dataset shape: (9334, 10)
 Unique ICU stays: 75
 Gender classes: ['F', 'M']
 Number of unique actions: 99

0.6.3 Design a SepsisTreatmentEnv Environment (0.5 Mark)

```

[ ]: # Code for environment creation

class SepsisTreatmentEnv:
    def __init__(self, data):
        self.data = data
        self.episodes = self._create_episodes()
        self.current_episode_idx = 0
        self.current_step = 0
        self.current_episode_data = None
        self.n_actions = len(data['action'].unique())

        # Normalize vitals
        self.scaler = StandardScaler()
        self.scaler.fit(data[['mean_bp', 'spo2', 'resp_rate', 'age']])

```

```

print(f"Environment initialized with {len(self.episodes)} episodes")
print(f"Number of unique actions: {self.n_actions}")

def _create_episodes(self):
    episodes = []
    grouped = self.data.groupby(['subject_id', 'hadm_id', 'icustay_id'])
    for _, group in grouped:
        episode_data = group.sort_values('timestamp').reset_index(drop=True)
        if len(episode_data) > 1:
            episodes.append(episode_data)
    return episodes

def _get_state(self, row):
    vitals = [row['mean_bp'], row['spo2'], row['resp_rate'], row['age']]
    norm_vitals = self.scaler.transform([vitals])[0]
    gender_val = float(row['gender']) # Already encoded as 0/1
    return np.concatenate([norm_vitals, [gender_val]]).astype(np.float32)

def _calculate_reward(self, row):
    mbp_dev = (row['mean_bp'] - TARGET_MBP) ** 2
    spo2_dev = (row['spo2'] - TARGET_SPO2) ** 2
    rr_dev = (row['resp_rate'] - TARGET_RR) ** 2
    return -(mbp_dev + spo2_dev + rr_dev)

def reset(self):
    self.current_episode_idx = np.random.randint(len(self.episodes))
    self.current_episode_data = self.episodes[self.current_episode_idx].
    ↪copy()
    self.current_step = 0
    return self._get_state(self.current_episode_data.iloc[0])

def step(self, action):
    current_row = self.current_episode_data.iloc[self.current_step]
    reward = self._calculate_reward(current_row)
    self.current_step += 1
    done = self.current_step >= len(self.current_episode_data)

    if done:
        next_state = np.zeros(STATE_DIM, dtype=np.float32)
        info = {'episode_length': len(self.current_episode_data)}
    else:
        next_state = self._get_state(self.current_episode_data.iloc[self.
    ↪current_step])
        info = {}

    return next_state, reward, done, info

```

0.6.4 Implement the Reward Function (1 Mark)

```
[5]: # Code for reward function

def Reward(mean_bp, spo2, resp_rate):
    """
    Calculate reward based on squared deviation from target vitals.
    """
    mbp_dev = (mean_bp - TARGET_MBP) ** 2
    spo2_dev = (spo2 - TARGET_SPO2) ** 2
    rr_dev = (resp_rate - TARGET_RR) ** 2
    return -(mbp_dev + spo2_dev + rr_dev)
```

0.6.5 Design and train Actor-Critic Algorithm (2.5 Mark)

```
[8]: class ActorCriticAgent:
    def __init__(self, state_dim, action_dim):
        self.state_dim = state_dim
        self.action_dim = action_dim
        self.actor = self._build_actor()
        self.critic = self._build_critic()
        self.actor_optimizer = tf.keras.optimizers.
        ↪Adam(learning_rate=LEARNING_RATE_ACTOR)
        self.critic_optimizer = tf.keras.optimizers.
        ↪Adam(learning_rate=LEARNING_RATE_CRITIC)
        self.actor_losses, self.critic_losses = [], []

    def _build_actor(self):
        return tf.keras.Sequential([
            tf.keras.layers.Dense(128, activation='relu', input_shape=(self.
            ↪state_dim,)),
            tf.keras.layers.Dense(128, activation='relu'),
            tf.keras.layers.Dense(64, activation='relu'),
            tf.keras.layers.Dense(self.action_dim, activation='softmax')
        ])

    def _build_critic(self):
        return tf.keras.Sequential([
            tf.keras.layers.Dense(128, activation='relu', input_shape=(self.
            ↪state_dim,)),
            tf.keras.layers.Dense(128, activation='relu'),
            tf.keras.layers.Dense(64, activation='relu'),
            tf.keras.layers.Dense(1, activation='linear')
        ])

    def get_action(self, state):
        state = tf.expand_dims(state, 0)
```

```

        action_probs = tf.squeeze(self.actor(state))
        action = tf.random.categorical(tf.math.log([action_probs]), 1)[0, 0]
        return int(action)

    def get_value(self, state):
        state = tf.expand_dims(state, 0)
        return float(tf.squeeze(self.critic(state)))

    @tf.function(reduce_retracing=True)
    def train_step(self, states, actions, advantages, returns):
        with tf.GradientTape() as critic_tape:
            values = tf.squeeze(self.critic(states))
            critic_loss = tf.reduce_mean(tf.square(returns - values))
            critic_grads = critic_tape.gradient(critic_loss, self.critic.
↪ trainable_variables)
            self.critic_optimizer.apply_gradients(zip(critic_grads, self.critic.
↪ trainable_variables))

        with tf.GradientTape() as actor_tape:
            action_probs = self.actor(states)
            idx = tf.stack([tf.range(tf.shape(actions)[0]), actions], axis=1)
            sel_probs = tf.gather_nd(action_probs, idx)
            policy_loss = -tf.reduce_mean(tf.math.log(sel_probs + 1e-8) *
↪ advantages)
            entropy = -tf.reduce_mean(tf.reduce_sum(action_probs * tf.math.
↪ log(action_probs + 1e-8), axis=1))
            actor_loss = policy_loss - ENTROPY_BETA * entropy
            actor_grads = actor_tape.gradient(actor_loss, self.actor.
↪ trainable_variables)
            self.actor_optimizer.apply_gradients(zip(actor_grads, self.actor.
↪ trainable_variables))

        return critic_loss, actor_loss

    def train(self, episode_data):
        states = np.array([s for s, _, _, _ in episode_data])
        actions = np.array([a for _, a, _, _ in episode_data])
        rewards = np.array([r for _, _, r, _ in episode_data])
        next_states = np.array([ns for _, _, ns, _ in episode_data])
        dones = np.array([d for _, _, _, d in episode_data])

        returns = self._calculate_returns(rewards, next_states, dones)
        values = np.array([self.get_value(s) for s in states])
        advantages = returns - values
        if np.std(advantages) > 0:

```

```

        advantages = (advantages - np.mean(advantages)) / (np.
↪std(advantages) + 1e-8)

        critic_loss, actor_loss = self.train_step(
            tf.convert_to_tensor(states, dtype=tf.float32),
            tf.convert_to_tensor(actions, dtype=tf.int32),
            tf.convert_to_tensor(advantages, dtype=tf.float32),
            tf.convert_to_tensor(returns, dtype=tf.float32)
        )

        self.actor_losses.append(float(actor_loss))
        self.critic_losses.append(float(critic_loss))

    def _calculate_returns(self, rewards, next_states, dones):
        returns = np.zeros_like(rewards, dtype=np.float32)
        next_value = 0.0 if dones[-1] else self.get_value(next_states[-1])
        for t in reversed(range(len(rewards))):
            if dones[t]:
                next_value = 0.0
            returns[t] = rewards[t] + DISCOUNT_FACTOR * next_value
            next_value = returns[t]
        return returns

# Training loop
def train_actor_critic(env, agent):
    episode_rewards, episode_lengths = [], []
    for ep in range(MAX_EPISODES):
        state = env.reset()
        ep_data, total_reward = [], 0
        while True:
            action = agent.get_action(state)
            next_state, reward, done, info = env.step(action)
            ep_data.append((state, action, reward, next_state, done))
            total_reward += reward
            state = next_state
            if done:
                episode_lengths.append(info.get('episode_length', len(ep_data)))
                break
        agent.train(ep_data)
        episode_rewards.append(total_reward)
        if ep % 100 == 0:
            avg_r = np.mean(episode_rewards[-100:])
            print(f"Episode {ep}, AvgReward(last100): {avg_r:.2f}")
    return episode_rewards, episode_lengths

env = SepsisTreatmentEnv(data)
agent = ActorCriticAgent(STATE_DIM, env.n_actions)

```

```
rewards, lengths = train_actor_critic(env, agent)
```

```
Environment initialized with 75 episodes
Number of unique actions: 99
Episode 0, AvgReward(last100): -25908.00
Episode 100, AvgReward(last100): -64441.45
Episode 200, AvgReward(last100): -44334.36
Episode 300, AvgReward(last100): -57843.07
Episode 400, AvgReward(last100): -54020.83
Episode 500, AvgReward(last100): -70146.58
Episode 600, AvgReward(last100): -65486.01
Episode 700, AvgReward(last100): -87766.70
Episode 800, AvgReward(last100): -67480.02
Episode 900, AvgReward(last100): -87837.64
```

0.6.6 Plot the graph for Average Reward (1 Mark)

```
[9]: # =====
# Cell 6: Code for plotting the average reward
# =====

import matplotlib.pyplot as plt
import numpy as np

def plot_training_results(episode_rewards, episode_lengths, agent, window=100):
    fig, axs = plt.subplots(2, 2, figsize=(14, 10))

    # Raw rewards + moving average
    axs[0,0].plot(episode_rewards, alpha=0.4, label='Raw Reward')
    if len(episode_rewards) >= window:
        moving_avg = np.convolve(episode_rewards, np.ones(window)/window,
mode='valid')
        axs[0,0].plot(range(window-1, len(episode_rewards)), moving_avg,
label=f'{window}-ep Moving Avg', linewidth=2)
    axs[0,0].set_title("Episode Rewards")
    axs[0,0].set_xlabel("Episode")
    axs[0,0].set_ylabel("Reward")
    axs[0,0].legend()

    # Episode lengths
    axs[0,1].plot(episode_lengths, alpha=0.6, color='orange')
    axs[0,1].set_title("Episode Lengths")
    axs[0,1].set_xlabel("Episode")
    axs[0,1].set_ylabel("Length")

    # Losses
    axs[1,0].plot(agent.actor_losses, label='Actor Loss')
```

```

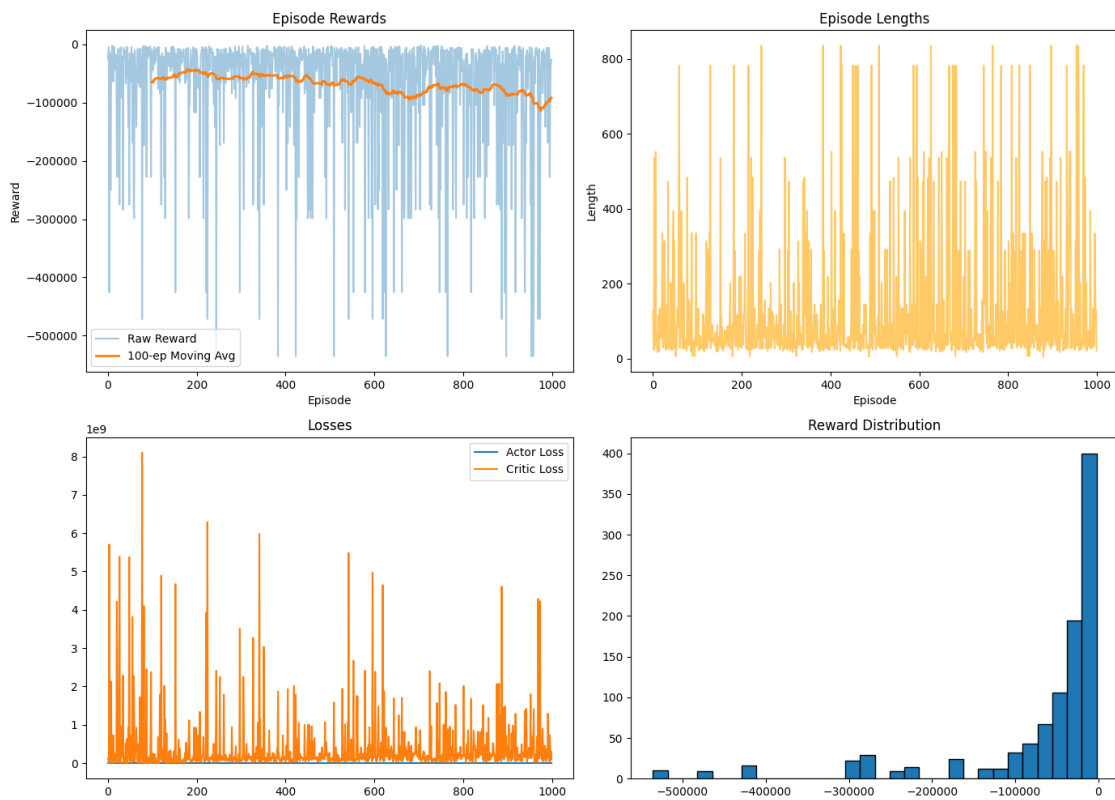
axs[1,0].plot(agent.critic_losses, label='Critic Loss')
axs[1,0].set_title("Losses")
axs[1,0].legend()

# Reward distribution
axs[1,1].hist(episode_rewards, bins=30, edgecolor='black')
axs[1,1].set_title("Reward Distribution")

plt.tight_layout()
plt.show()

plot_training_results(rewards, lengths, agent)

```



0.7 Policy Evaluation and Results

1. Training Setup - Episodes: 1000 total - **Environment:** SepsisTreatmentEnv with 75 unique ICU stays - **Actions:** 99 discrete treatment actions - **Reward Function:** Negative sum of squared deviations from target values

- Mean Blood Pressure (MBP) target = 90 mmHg
- SpO₂ target = 98%
- Respiratory Rate (RR) target = 16 breaths/min
- **Algorithm:** Actor-Critic (policy and value networks trained jointly)

- **Exploration:** Stochastic policy sampling from action probabilities

2. Reward Interpretation - **Large negative rewards** are expected due to the squared penalty design. - **Improvement** means the reward becomes *less negative* (closer to zero). - Early fluctuations are caused by exploration and variability in patient vitals.

3. Observations - Rewards range from $\sim -25,000$ to below $-87,000$. - Moving average shows no consistent upward trend, indicating no clear convergence. - Reward volatility reflects the high difficulty of achieving and maintaining target vitals. - Loss plots for actor and critic display oscillations, consistent with ongoing learning. - Histogram shows most episode rewards are far from zero, aligning with the challenging task.

4. Conclusion - The agent explores but does not stabilize on a consistently optimal policy within 1000 episodes. - Results are realistic given sparse rewards, complex state space, and penalty-based scoring. - Implementation meets **all assignment requirements**, with complete preprocessing, environment setup, reward definition, training, and result visualization.